SOFTWARE ENGINEERING

Chapter 7: Understanding Requirements

Based on: Software Engineering: A Practitioner's Approach (9th Edition)

Authors: Roger S. Pressman and Bruce R. Maxim

Student Name: Asal Ghafouri
Instructor: Engineer Rasoulzadeh
Course: Software Engineering

1. Introduction

Understanding what the customer really needs is one of the most critical and difficult activities in software engineering. Many projects fail not because of poor coding, but because the requirements were incomplete, ambiguous, or misunderstood.

Chapter 7 of Pressman and Maxim's book focuses on Requirements Engineering — the systematic process of discovering, analyzing, documenting, validating, and managing the requirements of a software system.

This chapter explains the main tasks of requirements engineering, the common problems that arise, and practical techniques for gathering and refining requirements. Examples are based on the continuing online shopping application project.

2. Common Problems with Requirements Practices

In many projects, requirements work suffers from recurring problems:

1. Difficulty understanding the requirements that customers provide.
2. Recording requirements in a disorganized or incomplete manner.
3. Spending too little time verifying what has been recorded.
4. Allowing uncontrolled change to dominate the project instead of managing it.
5. Failing to establish a solid foundation for the system the user actually wants.

Requirements engineering provides a structured way to reduce these problems.

3. Requirements Engineering Tasks

Requirements engineering consists of a set of related activities that begin during communication and continue into modeling:

1. Inception – Establish a basic understanding of the problem, the people who want a solution, and the nature of the desired solution. Effective customer–developer communication starts here.
2. Elicitation – Draw out requirements and business goals from all stakeholders.
3. Elaboration – Develop a refined requirements model that identifies functions, behavior, and information.
4. Negotiation – Reach agreement on a realistic scope that both developers and customers can accept.

5. Specification – Produce written documents, models, scenarios, or prototypes that describe the requirements.
6. Validation – Assess the quality and consistency of the requirements work products.
7. Requirements Management – Track requirements and their changes throughout the project.

4. Establishing the Groundwork

Before detailed gathering begins, the team must establish a solid foundation:

4.1 Identifying Stakeholders

Stakeholders are anyone who has an interest in the system — customers, end users, managers, marketing staff, support personnel, and others. Identifying all relevant stakeholders early prevents later surprises.

4.2 Recognizing Multiple Viewpoints

Different stakeholders often have different and sometimes conflicting needs. The team must recognize these multiple viewpoints and work to reconcile them.

4.3 Working Toward Collaboration

Successful requirements work depends on collaboration between customers and developers. Both sides must be willing to listen, negotiate, and compromise.

4.4 Asking the First Questions

Early questions help clarify the context of the project: Who is behind the request? Who will use the solution? What is the economic benefit? Are there other solutions under consideration?

4.5 Nonfunctional Requirements

In addition to functional requirements (what the system must do), nonfunctional requirements address quality attributes such as performance, security, usability, reliability, and constraints.

4.6 Traceability

Traceability links requirements to their sources and to later design and test artifacts. It helps the team understand the impact of changes.

5. Requirements Gathering

Gathering is the activity of collecting information from stakeholders. Effective techniques include:

5.1 Collaborative Requirements Gathering

Workshops or facilitated meetings bring stakeholders and developers together. Techniques such as brainstorming, prioritization, and consensus building are used to produce a shared understanding.

5.2 Usage Scenarios

Usage scenarios (or user stories) describe how a user interacts with the system to achieve a goal. They help make abstract requirements concrete.

Example:

> *"As a registered customer, I want to add products to my shopping cart so that I can purchase several items in one transaction."*

5.3 Elicitation Work Products

Typical work products include lists of stakeholders, prioritized feature lists, usage scenarios, constraints, and initial nonfunctional requirements.

6. Developing Use Cases

Use cases describe interactions between actors (users or external systems) and the system under development. A use case typically includes:

- A name and brief description
- Primary actor
- Preconditions and postconditions
- Main success scenario
- Alternative flows and exceptions

Example – Use Case: Place Order

Primary Actor: Registered Customer

Preconditions: Customer is logged in and has items in the cart.

Main Success Scenario: Customer reviews cart → selects payment method → confirms order → system processes payment and creates order record → customer receives confirmation.

Exceptions: Payment fails, inventory is insufficient, network timeout.

7. Building the Analysis Model

The analysis model refines the requirements into a more structured representation. It typically includes:

- Scenario-based elements (use cases, user stories)
- Class-based elements (classes and relationships)
- Behavioral elements (state diagrams, sequence diagrams)
- Flow-oriented elements (data flow or process models)

Analysis patterns can also be reused to solve recurring problems in a consistent way.

8. Negotiating Requirements

Customers often request more features than can be delivered within time and budget. Negotiation is the process of reaching a realistic and mutually acceptable set of requirements.

Effective negotiation requires:

- Clear prioritization of features
- Honest discussion of constraints
- Willingness on both sides to make trade-offs

Example:

The customer wants advanced product recommendations and a loyalty program in the first release. After discussion, the team and customer agree to deliver basic browsing, cart, and payment first, and to schedule the advanced features for a later release.

9. Validating and Managing Requirements

9.1 Validating Requirements

Validation checks that the requirements are correct, complete, consistent, and clear. Techniques include reviews, walkthroughs, and prototyping. The goal is to find problems before design and coding begin.

9.2 Requirements Management

Requirements change throughout a project. Requirements management includes:

- Identifying and documenting requirements
- Controlling changes through a defined process
- Tracking the status of each requirement
- Maintaining traceability between requirements and other artifacts

10. Practical Example: Online Shopping Application

Applying the ideas of this chapter to the online shopping project:

10.1 Inception and Stakeholders

Stakeholders identified: business owner, marketing team, end customers, customer support staff, and the development team.

10.2 Elicitation

Through workshops and interviews, the team gathers features such as product browsing, search, user accounts, shopping cart, payment, order tracking, and basic reviews.

10.3 Use Cases and Scenarios

Key use cases are written: Browse Products, Register Account, Add to Cart, Checkout, Track Order. Each includes main and alternative flows.

10.4 Negotiation

Advanced recommendation engine and multi-language support are deferred to later releases so that the first version can be delivered on schedule.

10.5 Validation and Management

Requirements are reviewed with stakeholders. A simple tracking sheet is used to record changes and their impact as the project progresses.

11. Conclusion

Chapter 7 emphasizes that understanding requirements is a disciplined engineering activity, not an informal conversation. By following the tasks of inception, elicitation, elaboration, negotiation, specification, validation, and management, teams can build a solid foundation for the software they will design and construct.

Clear requirements reduce rework, improve communication, and increase the chance that the final product will satisfy real user needs. The online shopping application example shows how these activities can be applied in a realistic project setting.

12. References

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Wiegers, K., & Beatty, J. (2013). *Software Requirements* (3rd ed.). Microsoft Press.